Conversational Modeling and Decoding Strategies

Author: Lam Nguyen

Available: March 10, 2026 12:00am until March 31, 2026 11:59pm

This assignment examines conversational modeling, conversational question answering, and decoding strategies for language models. All questions must be answered strictly using the material covered in this module. No external sources are required or permitted.

This is an individual assignment. Answers must be technically precise, clearly structured, and grounded in the concepts presented in class.

Question 1: Multi-Turn Dialogue Modeling

Question: Explain how multi-turn dialogue modeling conditions a response on prior conversational turns. Define the conditional probability objective used for dialogue generation and describe how conversational history is incorporated into the model input.

Answer: Multi-turn dialogue modeling creates responses by conditioning on the entire conversational history, which allows coherence and context across the entire chat.

The conditional probability objective is typically formulated as P(yt|x1,y1,x2,y2,...,xt)$P(yt|x1,y1,x2,y2,...,xt)$, where ytyt is the current response and x1,y1,...,xt$x1,y1,...,xt$ represent the previous user's statements and the model's responses.

Conversational history is incorporated by concatenating prior turns as a single sequence input to the model, often separated by tokens like [SEP] or [USER]/[SYSTEM], allowing the encoder or decoder to attend to relevant context when generating each token of the response. This allows the model to have a coherent conversation where past information is remembered and incorporated in responses.

Question 2: Dialogue State Tracking

Question: Define dialogue state tracking. Describe what constitutes the dialogue state, how it is updated across turns, and how it supports coherent task-oriented dialogue.

Answer: DST is the process of monitoring and maintaining a structured representation of the user's goals, constraints, and preferences throughout a conversation. The dialogue state consists of slot-value pairs representing domain-specific information such as dates, locations, etc..., along with the current dialogue act and user intent.

This state is updated after each user's input by extracting new information, modifying existing slots, or confirming values through natural language components. DST supports coherent task-oriented dialogue by providing a persistent memory of what has been discussed, enabling the system to track partial information across turns, handle clarifications and corrections, generate contextually relevant system prompts, and ultimately fulfill the user's request by maintaining a complete picture of their requirements throughout the conversation.

Question 3: Conversational Question Answering

Question: Define conversational question answering. Explain how conversational history is incorporated to resolve context-dependent questions, including ellipsis and coreference.

Answer: Conversational question answering is a task where the model answers a sequence of related questions by using the conversational context, rather than treating each question independently. The system uses conversational history by encoding previous question-answer pairs along with the current question and context document.

This enables the model to resolve anaphoric references ("What about them?"), handle ellipsis ("Her hobbies?"), and maintain topic continuity across the conversation. This is typically implemented by concatenating the dialogue history with the current question as input to a reading comprehension model.

Another method is using explicit coreference resolution and question rewriting modules. These modules transform context-dependent questions into standalone ones by substituting pronouns and completing elliptical phrases based on previous turns. This allows the model to ground references and provide accurate answers even when questions lack obvious context.

Question 4: Extractive vs. Generative Conversational Models

Question: Compare extractive and generative approaches to conversational question answering. Describe their architectural differences and explain how their output generation mechanisms differ.

Answer: Extractive conversational question answering is a method that identifies and extracts answer spans directly from a given context document by predicting start and end positions of the answer within the source text. This method uses architectures like BERT with span prediction heads that output two distributions over token positions.

Generative conversational models produce answers by generating tokens sequentially using encoder-decoder architectures (like T5 or BART) or decoder-only models (like GPT), where the model learns to synthesize responses token after token (autoregressive) conditioned on the question and context.

The architectural differences between the two lies in the output mechanism: extractive models use classification heads to select existing spans, while generative models use a language modeling head with a vocabulary-sized softmax to produce free-form text, enabling them to paraphrase, summarize, or generate answers not explicitly present in the source, although this can cause hallucinations.

Question 5: Sequence Generation Objective

Question: Write and explain the conditional probability objective used for sequence generation. Explain how next-token prediction leads to full sequence generation.

Answer: The conditional probability objective for sequence generation is formulated as P(y|x)=∏t=1TP(yt|y<t,x)$P(y|x)=\prod_{t=1}^{T}P(y_t|y_{<t},x)$, where x$x$ is the input sequence (e.g., context or prompt), y=(y1,y2,...,yT)$y=(y_1,y_2,...,y_T)$ is the target output sequence, and y<t$y_{<t}$ reps all previous tokens. This objective decomposes the probability of generating the entire sequence into a product of conditional probabilities for each token given all previous tokens and the input.

During training, the model maximizes this likelihood by learning to predict each token given the ground truth history, while during inference, next-token prediction leads to full sequence generation through autoregressive decoding. First the model predicts the first token y1y_1 conditioned on the input xx. Then it predicts y2y_2 conditioned on both xx and y1y_1, and continues iteratively until an end-of-sequence token is generated or a maximum length is reached.

Question 6: Greedy Decoding and Beam Search

Question: Describe greedy decoding and beam search. Explain how beam search improves upon greedy decoding and how beam width affects the search process and output selection.

Answer: Greedy decoding is the simplest generation strategy where at each timestep the model selects the single highest-probability token according to argmaxytP(yt|y<t,x)$\arg\max_{y_t} P(y_t|y_{<t},x)$, making locally optimal choices without considering future implications. Greedy decoding can lead to suboptimal overall sequences.

Beam search improves upon this by maintaining a fixed number of kk candidate sequences (the beam width) at each step, expanding each candidate by considering the top tokens, scoring complete partial sequences by their cumulative log-probability, and keeping only the kk highest-scoring sequences for the next iteration.

The beam width directly controls the exploration-exploitation tradeoff: larger beams (e.g., k=10$k=10$) explore more diverse hypotheses and are more likely to find globally better sequences but require more computation, while smaller beams (e.g., k=1$k=1$, equivalent to greedy) are faster but more prone to local optima.

At the end of generation, the sequence with the highest overall probability among the surviving beams is selected as the final output. This typically results in more coherent texts than greedy decoding.

Question 7: Sampling-Based Decoding

Question: Explain temperature scaling, top-k sampling, and nucleus (top-p) sampling. Describe how each method modifies the probability distribution during decoding and how this influences output diversity.

Answer: Temperature scaling modifies the probability distribution by dividing logits by a temperature parameter τττ before applying softmax: P(yt)=exp(zt/τ)∑jexp(zj/τ)$P(y_t)=\frac{\exp(z_t/\tau)}{\sum_j \exp(z_j/\tau)}$. Higher temps (τ>1$\tau>1$) flatten the distribution increasing randomness and diversity. Lower temps (τ<1$\tau<1$) sharpen it making high-probability tokens more dominant.

Top-k sampling restricts sampling to only the k*k* most probable tokens at each step, setting all other probabilities to zero and renormalizing. This prevents the model from selecting very unlikely tokens while maintaining diversity among the top candidates.

Nucleus (top-p) sampling dynamically selects the smallest set of tokens whose cumulative probability exceeds threshold p*p* (e.g., 0.9). It then samples from this set, adapting the number of candidates based on the model's confidence: when the model is certain, fewer tokens are considered, and when uncertain, more options are included. This provides more contextually appropriate diversity control than fixed top-k and generally producing more natural, varied outputs than deterministic methods while avoiding the gibberish of pure random sampling.

Question 8: Decoding and Output Behavior

Question: Explain how decoding strategy affects generated output behavior. Discuss its influence on diversity, repetition, and variability in generated text.

Answer: The decoding strategy shapes the characteristics of generated text by controlling the balance between quality, diversity, and creativity in the output. Deterministic methods like greedy decoding and beam search tend to produce safe, high-probability sequences that are often fluent and coherent but can suffer from repetition (especially in longer texts), generic phrasing, and lack of creativity. This is because they consistently favor the same high-likelihood paths.

Sampling-based methods introduce stochasticity that increases output diversity and variability, with higher temperatures or larger top-p values producing more creative and varied text. The tradeoff to this novelty is risking incoherence or factual errors, while lower temperatures or smaller sampling windows maintain quality but reduce novelty.

Repetition is particularly affected by the strategy: beam search can create repetition loops when the same n-grams receive high scores repeatedly, while sampling methods naturally avoid repetition through randomness but may still create semantic or structural repetition patterns.

The choice of decoding strategy is important for matching generation behavior to the specific application requirements: prioritizing factual accuracy and consistency (beam search), creative variety (sampling with higher temperature), or a compromise (nucleus sampling with moderate p values).